

Github Link - <https://github.com/TammyBriggs/alu-logo-asset-tokenization>

Demo Video -  Cryptocurrency and Digital Assets Demo Video

Abstract

This project involves the development of a blockchain-based security suite to register, protect, and fractionalize the official African Leadership University (ALU) logo. By leveraging Ethereum smart contracts, the system creates an immutable record of the logo's authenticity using its cryptographic hash. Furthermore, it introduces an ERC-20 tokenization model to divide the intellectual property into tradable shares, solving issues of digital provenance and decentralized ownership.

Introduction

ALU requires a robust mechanism to protect its official logo from unauthorized modification and false claims. While standard databases can store files, they are centralized and vulnerable to internal tampering or single points of failure. Traditional copyright registration is often slow, geographically limited, and lacks real-time, public verifiability. By anchoring the logo on the Ethereum blockchain, ALU establishes a mathematically verifiable, globally accessible, and immutable proof of origin. This ensures that anyone can instantly verify the authenticity of the logo without relying on a central authority, while preventing any alterations to the ownership record.

Question 1: Explore a Cryptocurrency Exchange Platform

Step 1: Account Creation & Identity Verification (KYC)

When registering an account on Coinbase, the platform requires standard registration data (legal name, email address, and a secure password). However, before any trading can occur, users must undergo a strict Identity Verification process. This requires uploading a government-issued identification document (such as a passport or national ID card) and providing a real-time facial selfie to match the document. Additionally, the platform asks for residential address verification and basic questions regarding the user's source of funds and employment.

Exchanges require this because of **Know Your Customer (KYC)** and **Anti-Money Laundering (AML)** regulations. These frameworks prevent financial institutions from being used for illicit activities, money laundering, or terrorist financing. Globally, these actions are mandated by bodies like the **Financial Action Task Force (FATF)** and enforced locally by national financial authorities to ensure accountability within the digital asset ecosystem.

Step 2: Platform Exploration & Asset Analysis

Navigating the interface reveals three main areas: the **Dashboard** (a macro view of account balances and market movements), the **Wallet** (where private keys are custodied by the exchange to store assets), and the **Trading Area** (the order book and charting interface).

Three major cryptocurrencies listed on the platform include:

- **Bitcoin (BTC):** The pioneer digital asset operating on a Proof-of-Work consensus mechanism. It is primarily used as a decentralized store of value ("digital gold") due to its hardcoded maximum supply cap of 21 million coins, distinguishing it from programmatic utility platforms.
- **Ethereum (ETH):** A decentralized, open-source blockchain featuring smart contract functionality. Unlike Bitcoin, ETH acts as the "gas" or transactional fuel required to execute decentralized applications (dApps) and run programmable logic across its global network.
- **USD Coin (USDC):** A stablecoin issued by Circle that is pegged 1:1 with the United States Dollar. It is backed by fully reserved assets, distinguishing it completely from BTC and ETH by offering price stability for low-volatility transfers and collateralization.

22:50



Search



7



Cash

\$0.00



Prices

Watchlist ▾



Ethereum

2.26% APY



\$2,377.76

▾ 2.21%



Bitcoin

BTC



\$60,543.79

▾ 1.85%



XRP

XRP



\$0.53

▴ 1.27%



Bitcoin Cash

BCH



\$321.98

▾ 0.53%



Litecoin

LTC



\$64.04

▾ 1.33%

Step 3: Adding a Payment Method

To fund an account, users navigate to the payment settings to link a financial source. Adding a debit card requires the card number, expiration date, CVV, and billing address. Adding a bank account requires routing numbers, account numbers, or online banking authentication via secure portals like Plaid.

13:58



Add a payment method



Bank account

Invest large amounts

Use to buy crypto and deposit or withdraw cash.

Recommended



Debit card

Invest small amounts

Use to buy crypto or deposit cash.



PayPal

Invest large amounts

Use to buy crypto, deposit and withdraw cash.



Direct deposit

Get your paycheck deposited into Coinbase as crypto or US dollars.

New



Wire transfers

Invest large amounts, no limits

Use to deposit or withdraw cash.

- **Debit Cards** are significantly faster, offering near-instantaneous settlement of crypto purchases. However, they carry higher convenience fees (often 3.99%).
- **Bank Transfers (ACH/Wire)** are much cheaper (frequently free or a low flat fee), but they take 3 to 5 business days to clear before the funds are fully settled.

Step 4: Simulated \$100 Purchase

When preparing a \$100 purchase order, users choose between two primary order types:

- **Market Order:** Executes immediately at the best available current market price.
- **Limit Order:** Sets a specific maximum price (for a buy) or minimum price (for a sell) at which the trade can execute. It only executes if the market hits that designated price point.

For this simulation, a **Market Order** is selected to guarantee immediate execution. If purchasing Ethereum at a hypothetical market price of \$3,400 with a standard flat transaction fee of \$2.99, the net investment amount is \$97.01.

The calculation for the amount received is:

$$\text{Asset Amount} = \frac{\text{Net Investment}}{\text{Market Price}}$$

$$\text{ETH Received} = \frac{\$97.01}{\$3,400} \approx 0.028532 \text{ ETH}$$

Step 5: Transaction Records & Hashes

The transaction history logs the transaction date, time, asset type, order type, fees paid, and total fiat value. Crucially, each finalized ledger entry includes a **Transaction Hash (TxID)**. A transaction hash is a unique, alphanumeric string generated by passing the transaction details through a cryptographic hashing function. It serves as an immutable digital receipt, allowing anyone to verify the status, validity, and completion of the transfer publicly on a blockchain explorer.

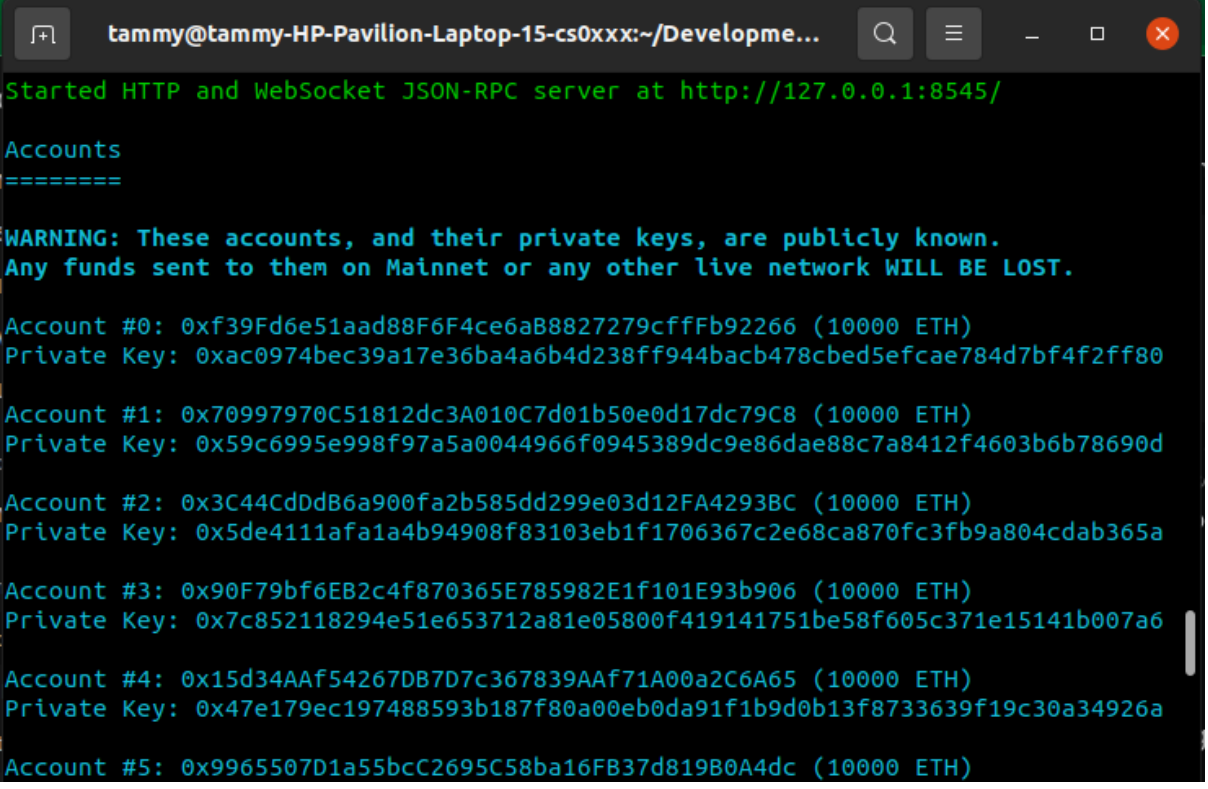
Step 6: Robust Security Features

To safeguard user capital, platforms implement institutional-grade security measures:

- **Two-Factor Authentication (2FA):** Requires a secondary time-sensitive token from an authenticator app (like Google Authenticator) alongside a password. This prevents account takeovers even if an attacker compromises the primary password.
- **Address Whitelisting / Withdrawal Vaults:** Restricts asset withdrawals to pre-approved addresses and enforces a 48-hour delay period on new addresses. This provides a critical time window to cancel unauthorized transfers if account credentials are breached.

Breakdown of the Hardhat Environment Setup

To set up the local blockchain environment, I first installed Node.js to manage the project dependencies. I then initialized a new Hardhat project via my terminal using `npm install --save-dev hardhat` and `npx hardhat init`. Once configured, I started a local blockchain node which generated 20 test wallet addresses pre-funded with 10,000 fake ETH to use for gas-free deployment testing.

A terminal window with a dark background and light green text. The title bar shows the user 'tammy' on a machine named 'tammy-HP-Pavilion-Laptop-15-cs0xxx' in the directory '~/Developme...'. The terminal output shows the start of an HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/. It then lists 20 accounts, with the first five shown in detail. Each account entry includes the account number, its public address, the amount of ETH (10000), and its private key. A warning message states that these accounts and their private keys are publicly known and that funds sent to them on Mainnet or any other live network will be lost.

```
tammy@tammy-HP-Pavilion-Laptop-15-cs0xxx:~/Developme...  
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/  
  
Accounts  
=====
```

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.

Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80

Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Private Key: 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d

Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
Private Key: 0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a

Account #3: 0x90F79bf6EB2c4f870365E785982E1f101E93b906 (10000 ETH)
Private Key: 0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6

Account #4: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65 (10000 ETH)
Private Key: 0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a

Account #5: 0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc (10000 ETH)

To understand how our local blockchain environment operates, we must examine the roles of the core configuration files and directories generated during initialization:

- **hardhat.config.js**: This is the master configuration file for the entire project. It specifies the Solidity compiler version, defines local and remote network configurations (such as the local network running on port 8545), and manages installed plugins (like the Hardhat Toolbox). Any automation or deployment settings must be registered here.
- **contracts/**: This directory is the designated storage folder for all smart contract source files written in Solidity. The Hardhat compiler automatically scans this folder to locate and build your code.
- **scripts/**: This folder contains automation scripts written in JavaScript or TypeScript. These scripts are used to programmatically interact with your contracts, most notably for deploying your compiled contracts to a local or live blockchain network and executing post-deployment tasks.

- **test/:** This directory houses your automated testing suites. Writing comprehensive unit tests here ensures your smart contracts execute precisely as intended and behave securely before facing live network environments.
- **package.json:** A standard Node.js manifest file that records the project's metadata, external dependencies (such as Hardhat and OpenZeppelin libraries), and custom execution scripts.
- **.env & .gitignore:** These work together as a critical security perimeter. The .env file holds confidential keys (like private keys or API credentials) as environment variables so they aren't hardcoded into the scripts. The .gitignore file ensures that this sensitive file, along with large generated folders like node_modules/ and build artifacts, are never accidentally uploaded or exposed on public repositories like GitHub.

What is a local blockchain and why do developers use one instead of the real Ethereum network during development? A local blockchain is an isolated, simulated instance of the Ethereum network running entirely on a developer's machine (using tools like Hardhat). Developers use it because every transaction on the live mainnet costs real cryptocurrency (gas fees) and is permanent. A local environment provides a risk-free sandbox with fast transaction execution and infinite "test" funds, allowing developers to debug contracts, trace errors, and run automated testing suites at zero cost before deploying to a public network.

What is a wallet address and what does it represent on the blockchain? A wallet address is a unique, public-facing identifier string (starting with 0x on Ethereum) derived cryptographically from a private key. On the blockchain, it represents a specific account or "identity." It acts much like a bank account number, serving as the destination for receiving digital assets, interacting with smart contracts, and tracking the ownership and balances associated with that specific entity on the public ledger.

Smart Contract Explanation

What is a smart contract? How is it different from a normal programme running on a server? A smart contract is a self-executing program stored and running directly on a decentralized blockchain network. Unlike a traditional application deployed on a centralized server (like AWS or Google Cloud), a smart contract cannot be modified, turned off, or manipulated by a single administrator. It operates deterministically, meaning its rules are enforced identically by every node in the network, making its execution trustless, transparent, and completely audit-proof.

What is an NFT and why is ERC-721 the right standard for registering a unique asset like a logo? An NFT (Non-Fungible Token) is a unique digital item that cannot be directly interchanged with another token on a 1:1 basis, as each contains distinct characteristics and identifiers. The ERC-721 standard is the correct choice for registering the ALU logo because a corporate logo is a distinct, non-divisible piece of intellectual property. Unlike fungible currencies (where every dollar or bitcoin is identical), a unique logo file requires custom

metadata properties, individual provenance tracking, and absolute singular ownership records.

What is a SHA-256 hash and why does changing even one pixel of the logo produce a completely different hash? A SHA-256 hash is a cryptographic function that takes any input data (such as an image file) and maps it to a fixed-size 256-bit (32-byte) alphanumeric signature. Cryptographic hashing functions possess a property known as the "avalanche effect." Because the input data is processed through complex rounds of bitwise mathematical transformations, altering even a single pixel or bit changes the entire state matrix of the file during compilation, resulting in a fundamentally radically different output hash. This makes it impossible to tamper with a file without breaking its cryptographic signature.

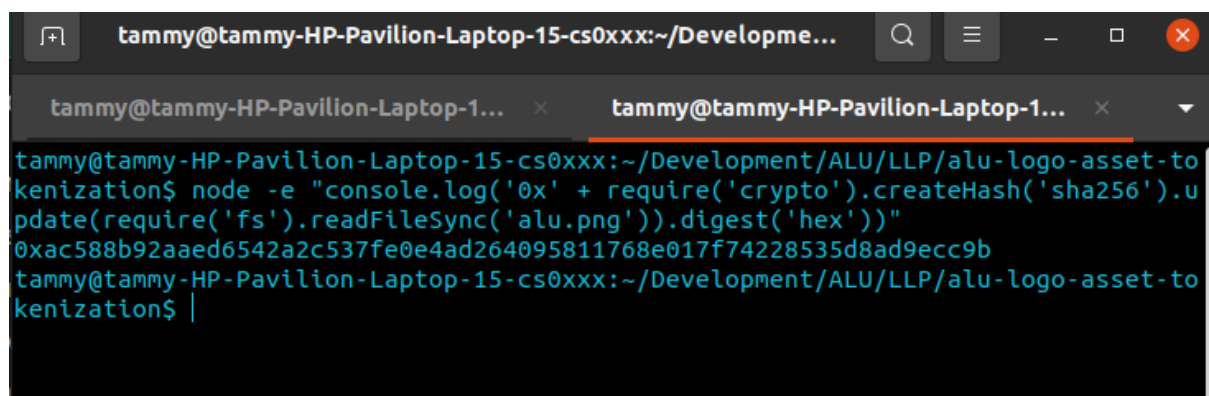
Why can the data not be changed once it is stored on the blockchain? Data on a blockchain is immutable because transactions are bundled into sequential blocks tied together using cryptographic hash chains. Once a block is written to the ledger, modifying any data within it would change that block's hash, invalidating every subsequent block in the chain. Changing recorded data would require a malicious actor to compromise and control a majority of the computing nodes across the decentralized network simultaneously to rewrite history, making past records practically unalterable.

Generating the ALU Logo Hash

To generate the unique fingerprint for the ALU logo, I used a free online SHA-256 hash generator. I uploaded the official `alu-logo.png` file into the tool, which processed the image data to output the 64-character hex string:

`0xac588b92aaed6542a2c537fe0e4ad264095811768e017f74228535d8ad9ecc9b`.

This string was then passed into the deployment script.

A terminal window screenshot showing a command being executed to generate a SHA-256 hash for the ALU logo. The command is: `node -e "console.log('0x' + require('crypto').createHash('sha256').update(require('fs').readFileSync('alu.png')).digest('hex'))"`. The output is the 64-character hex string: `0xac588b92aaed6542a2c537fe0e4ad264095811768e017f74228535d8ad9ecc9b`. The terminal window title is `tammy@tammy-HP-Pavilion-Laptop-15-cs0xxx:~/Development/ALU/LLP/alu-logo-asset-to-kenization$`.

Smart Contract Functions

- **registerAsset()**: This function accepts the asset name, file type, and the SHA-256 hash. It first checks a mapping to ensure the hash hasn't been registered before. If it is unique, it mints a new ERC-721 token to the deployer, saves the metadata to the blockchain, marks the hash as used, and returns a new token ID.
- **verifyLogoIntegrity()**: A read-only (zero-gas) function that takes a token ID and a user-provided hash. It compares the user's hash against the official hash

stored on the blockchain, returning `true` with an "authentic" message if they match, or `false` with a "warning" message if the file has been altered.

- **`getAsset()`**: A read-only function that retrieves the full `AssetMetadata` struct (name, file type, hash, registrar address, and timestamp) for a specific token ID so anyone can view the public registration record.
- **`distributeShares()`**: Located in the ERC-20 contract and restricted to the contract owner, this function transfers a specified amount of ALUT tokens from the owner's balance to a recipient's wallet to distribute fractional ownership.
- **`ownershipPercentage()`**: A read-only function that calculates a specific wallet's ownership stake. It divides their token balance by the total supply of 1,000,000 and multiplies by 100 to return a whole number percentage.

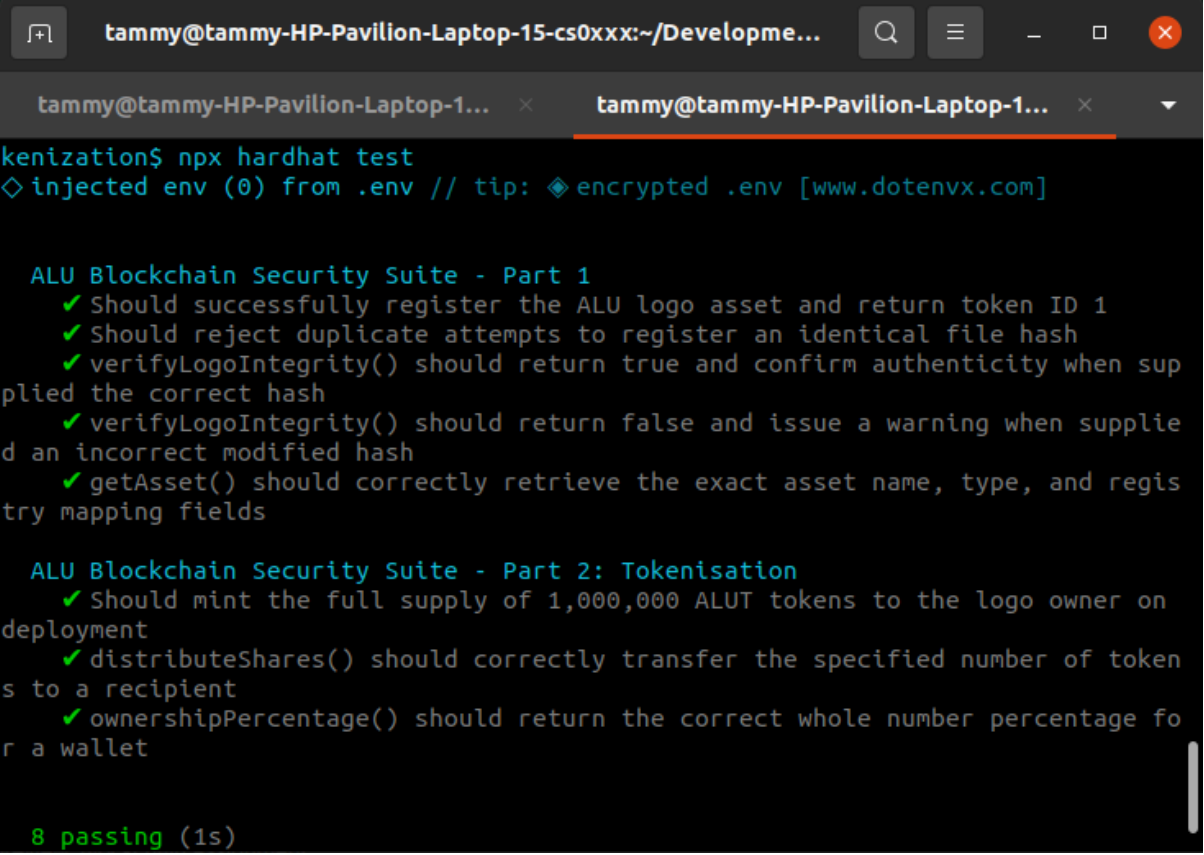
Tokenisation

What is tokenisation? Use the ALU logo as a specific example, who at ALU might receive tokens and why? Tokenisation is the process of converting the ownership rights of a distinct real-world or digital asset into fractional, fungible digital tokens on a blockchain. For the ALU logo, instead of the university holding a single static ownership record, they can issue 1,000,000 ALUT shares. ALU might distribute 51% of these tokens to the University Board of Directors to maintain majority governance, 20% to the marketing department to incentivize brand growth, and 29% to student alumni organizations as a programmable, tradable badge of community participation and vested interest in the university's brand equity.

What is the difference between ERC-721 and ERC-20? Why did you use ERC-721 for the logo registration but ERC-20 for the ownership shares? ERC-721 is the standard for Non-Fungible Tokens (NFTs); every token is mathematically unique and cannot be divided or treated equally with another token. ERC-20 is the standard for fungible tokens, where every token is exactly identical and can be subdivided, much like fiat currency. We used ERC-721 to anchor the physical logo because the intellectual property file is a singular, unique asset. We then layered an ERC-20 contract on top of it because the *financial ownership* of that singular asset needs to be divisible, identical, and liquid among multiple shareholders.

If a faculty member holds 100,000 ALUT tokens out of 1,000,000 total, what does that mean in practice? In practice, holding 100,000 out of 1,000,000 tokens means the faculty member owns exactly a 10% equitable stake in the intellectual property of the ALU logo. In a real-world scenario, if the university licenses the logo to an external apparel manufacturer for a royalty fee of \$10,000, the smart contract could be programmed to automatically read the token balances and instantly route \$1,000 (10%) directly to that faculty member's wallet as passive dividend income.

Test Results



```
tammy@tammy-HP-Pavilion-Laptop-15-cs0xxx:~/Developme...
tammy@tammy-HP-Pavilion-Laptop-1... x tammy@tammy-HP-Pavilion-Laptop-1... x
kenization$ npx hardhat test
◇ injected env (0) from .env // tip: ◇ encrypted .env [www.dotenvx.com]

ALU Blockchain Security Suite - Part 1
  ✓ Should successfully register the ALU logo asset and return token ID 1
  ✓ Should reject duplicate attempts to register an identical file hash
  ✓ verifyLogoIntegrity() should return true and confirm authenticity when supplied the correct hash
  ✓ verifyLogoIntegrity() should return false and issue a warning when supplied an incorrect modified hash
  ✓ getAsset() should correctly retrieve the exact asset name, type, and registry mapping fields

ALU Blockchain Security Suite - Part 2: Tokenisation
  ✓ Should mint the full supply of 1,000,000 ALUT tokens to the logo owner on deployment
  ✓ distributeShares() should correctly transfer the specified number of tokens to a recipient
  ✓ ownershipPercentage() should return the correct whole number percentage for a wallet

8 passing (1s)
```

Conclusion

Through this project, I gained practical experience in designing and deploying smart contracts using the Hardhat development environment. I learned how cryptographic hashing secures digital assets and how different token standards (ERC-721 and ERC-20) can interact to create a comprehensive digital property system. If I had more time, I would improve the system by adding a decentralized storage solution like IPFS to host the actual image file alongside its hash. I would also implement a Decentralized Autonomous Organization (DAO) contract so that ALUT token holders could actively vote on licensing decisions regarding the ALU logo.